



Facultad de Estudios Superiores Acatlán

Diplomado en Ciencia de Datos

Proyecto final · Módulo V



Señal de arrepentimiento temprano
antes de comprar un videojuego

Fernando Axel Ramírez Gómez

30 de septiembre de 2026

Repositorio: <https://github.com/fernandoaxelramirezgomez-coder/nexplay>

App en vivo: <https://nexplay-six.vercel.app>



Resumen

[PENDIENTE: redacción]



Índice

Resumen	1
1. Introducción	3
2. Planteamiento del problema	3
3. Nicho de negocio y usuario final	3
4. Estrategia: accionables, usuarios y usabilidad	4
5. Descripción de los datos	4
6. Definición del objetivo	8
7. Análisis exploratorio	10
8. Procesamiento de datos	14
9. Modelación	17
10. Evaluación y generalización	19
11. Interpretabilidad	21
12. Arquitectura y despliegue	23
13. Interfaz de uso	24
14. Resolución del problema	24
15. Limitaciones	24
16. Conclusiones	24
17. Siguiendo pasos	24
Referencias	25
A. Reproducibilidad	25

1. Introducción

1.1. Contexto

[PENDIENTE: redacción]

1.2. Preguntas del proyecto

[PENDIENTE: redacción]

1.3. Qué entrega el proyecto

[PENDIENTE: redacción]

1.4. Mapa del documento contra la rúbrica

[PENDIENTE: redacción]

2. Planteamiento del problema

2.1. La decisión antes de pagar

[PENDIENTE: redacción]

2.2. Por qué el promedio de reseñas no basta

[PENDIENTE: redacción]

2.3. Qué se puede observar y qué no

[PENDIENTE: redacción]

3. Nicho de negocio y usuario final

3.1. Una capa previa a la compra

[PENDIENTE: redacción]

3.2. El usuario, por comportamiento

[PENDIENTE: redacción]

3.3. Veteranos y novatos

[PENDIENTE: redacción]

4. Estrategia: accionables, usuarios y usabilidad

4.1. Accionables

[PENDIENTE: redacción]

4.2. Usuarios y necesidades

[PENDIENTE: redacción]

4.3. Recorrido: PAYDAY 3 contra Dead Space

[PENDIENTE: redacción]

4.4. Usabilidad

[PENDIENTE: redacción]

5. Descripción de los datos

Los datos son reseñas públicas de Steam y la ficha de cada juego en la misma tienda. En cada reseña, Steam guarda si el voto fue negativo y cuántos minutos llevaba jugados quien la escribió; con esas dos cosas se define la señal de arrepentimiento temprano (sección 6). El modelo se entrena con 83 juegos y 123,972 reseñas (data-v1), y el sitio sirve un catálogo de 123 juegos y 184,367 reseñas (data-v3). No se usó Kaggle ni ningún conjunto de terceros.

5.1. La API de Steam

La ingesta, el proceso que bajó los datos de Steam, usa dos servicios públicos de la tienda¹:

¹Una API (interfaz de programación) es una dirección web que devuelve datos en un formato que un programa puede leer, en lugar de una página pensada para personas. Estos dos servicios no piden clave.

- `appreviews`, por juego, da el voto, los minutos jugados al escribir y en total, el texto, las fechas y dos datos del autor: cuántos juegos tiene y cuántas reseñas ha escrito. Se piden las reseñas más recientes (`filter=recent`), en inglés (`language=english`) y de cualquier forma de compra, de 100 en 100, hasta un tope de 1,500 por juego. Valve documenta este servicio en Valve Corporation (s.f.-b). Al consultarla para este documento, la página lo marca como obsoleto y remite a `GetAppReviews` (Valve Corporation, s.f.-a). El proyecto no depende de volver a bajarlo: los datos quedaron congelados en releases (sección 5.2).
- `appdetails`, por juego, da el precio y el descuento en pesos mexicanos (`cc=mx`), los géneros, la fecha de lanzamiento, la descripción en español (`l=spanish`) y la nota de Metacritic². Valve no documenta este servicio de forma oficial.

La nota de Metacritic se toma de la tienda; el proyecto nunca consulta Metacritic. En los 40 títulos externos se comparó la nota guardada con la que mostraba la tienda el 21 de septiembre de 2026, y coinciden 40 de 40³. Cuando la tienda no muestra nota, esa ausencia se guarda tal cual y el modelo la usa como una variable más.

5.2. Releases y su verificación

Cada corte de datos se publica como un release de GitHub con un tag fijo⁴. Antes de usar un archivo, se compara su sha256 con el publicado⁵. La tabla 1 resume los tres releases. Con `data-v1` se toman todas las decisiones del modelo: las variables, el entrenamiento y los umbrales. El release `data-v2` suma 40 juegos que el modelo nunca ve, y `data-v3` es lo que sirve el sitio.

Ningún release guarda el identificador de Steam de quien escribió la reseña.

5.3. Periodo que cubren

Las reseñas de `data-v1` se bajaron el 14 de septiembre de 2026, y las de los juegos que agregó `data-v2`, hasta el 21 de septiembre de 2026, en UTC⁶. La ingesta pide las reseñas más recientes de cada juego hasta llegar al tope, y 81 de los 83 juegos lo alcanzan. Un juego popular llena el tope en pocos días y uno de nicho necesita años: la muestra de cada juego cubre de 4 a 1,252 días, con una mediana de 64.

La figura 1 cuenta las reseñas de `data-v1` por mes de publicación. Casi todas las barras se juntan a la derecha: el 96.1 % de las reseñas de entrenamiento es de julio de 2025 en

²Metacritic es un sitio que promedia las calificaciones de la crítica especializada en una nota de 0 a 100. Steam muestra esa nota en la página de algunos juegos.

³La comparación está en `docs/evidencia/verificacion-40-steam.csv`.

⁴Un release es una versión publicada del repositorio, con archivos adjuntos. Su tag es el nombre que la identifica, como `data-v1`, y no cambia: quien baje `data-v1` dentro de un año recibe exactamente los mismos archivos.

⁵El sha256 es una huella de 64 caracteres que se calcula sobre el contenido de un archivo; si cambia un solo byte, cambia la huella. La comparación la hace `descargar_verificado`, en `backend/despliegue/utilidades.py`.

⁶UTC es el tiempo universal coordinado, la hora de referencia mundial. La Ciudad de México va seis horas atrás.

Release	Publicado	Juegos	Reseñas	Base (MB)	Extracto (MB)	Para qué
data-v1	2026-09-16	83	123,972	11.4	1.4	Entrenar, elegir las variables y fijar los umbrales.
data-v2	2026-09-21	123	184,367	16.7	1.9	Suma 40 juegos que el modelo no ve: la prueba externa.
data-v3	2026-09-27	123	184,367	16.7	—	data-v2 más los totales públicos de Steam por juego. Lo sirve la API.

Tabla 1: Releases de datos. La base es la copia reproducible de la base de datos (`nexplay_reproducible.db.xz`); el extracto, la tabla sin texto que lee el notebook 01 (`nexplay_extracto.parquet`). Los sha256 completos están en el anexo. Fuente: API de releases de GitHub y los archivos bajados.

adelante (97.1 % en el catálogo), y a la izquierda queda una cola delgada que llega hasta el 8 de abril de 2023.



Figura 1: Reseñas de data-v1 por mes de publicación. Fuente: release data-v1, columna `timestamp_created`.

La muestra retrata, entonces, el presente de cada juego. En un juego viejo, la señal temprana viene de quien lo compra tarde, en una oferta o después de una actualización. Esto es una limitación de la fuente (sección 15) y no se corrige con pesos, porque no hay forma de saber cómo era la mezcla de compradores en otra época.

5.4. Calidad

Antes de explorar, el notebook `00_exploracion` valida data-v1: tipos y nulos, llaves, rangos, textos repetidos y la partición de validación. La tabla 2 separa los chequeos que no encontraron nada de los que sí, con lo que se hizo en cada caso.

Las dos columnas que definen la señal, el voto y los minutos al reseñar, no tienen nulos

Chequeo (data-v1)	Resultado	Qué se hizo
<i>Sin hallazgos</i>		
Nulos en las reseñas	0 en 21 columnas	nada que corregir
Llaves: reseña repetida, juego sin reseñas, reseña sin juego	0 casos	nada que corregir
Rangos imposibles (6 reglas: minutos negativos, voto fuera de 0 y 1, fechas, precio, descuento, nota)	0 casos	nada que corregir
Nota de Metacritic contra la de la tienda hoy (40 juegos externos)	40 de 40 iguales	nada que corregir
<i>Hallazgos y qué se hizo</i>		
Más minutos al reseñar que en total ^a	1 caso	se conserva: la señal no cambia con ninguno de los dos contadores
Juegos de pago sin precio ^b	2: Grand Theft Auto V Legacy, New World: Aeternum	precio tomado como 0, con aviso en la ficha (§8)
Juegos sin nota de Metacritic	23 de 83	la falta de nota es una variable del modelo ^c
Textos repetidos de 8 palabras o más	359 en el mismo juego; 57 en 2 o más juegos	se marcan, no se borran (§8)
Reseñas plantilla	192 (0.15 %)	se marcan, no se borran
Perfil privado: aparece con 0 juegos ^d	59.9 % de las reseñas	es una bandera de privacidad, no una biblioteca vacía

Columnas: ^a playtime_at_review y playtime_forever; ^b es_gratis y precio_final; ^c metacritic_disponible; ^d num_games_owned.

Tabla 2: Chequeos de calidad sobre data-v1. Fuente: las funciones de backend/analisis/exploracion.py que usa el bloque de validación del notebook 00.

ni valores fuera de rango, y todas las variables del juego que usa el modelo se conocen antes de comprar. Tres hallazgos tocan al modelo, y ninguno se rellena en silencio. El precio de los 2 juegos de pago que llegaron sin precio se toma como 0, y la ficha lo avisa. La falta de nota de los 23 juegos sin Metacritic entra al modelo como una variable propia. Y el 0 de los perfiles privados⁷, que son el 59.9 % de las reseñas, se trata como una bandera de privacidad y no como una biblioteca vacía. Los textos repetidos y las plantillas se marcan y se conservan; la sección 8 explica por qué.

El tope de 1,500 reseñas por juego es una decisión de diseño: así cada título pesa casi lo mismo en el entrenamiento, y unos cuantos juegos muy populares no dominan la muestra. Tiene otra consecuencia, los empates de tamaño entre juegos, que la sección 9.3 trata al hablar de la validación.

⁷En Steam, cada persona decide si su perfil es público. Un perfil privado oculta la biblioteca, y la API lo reporta con 0 juegos.

5.5. Por qué estos datos sirven para la pregunta

NexPlay quiere saber, antes de la compra, qué tan seguido un juego deja a sus compradores arrepentidos en las primeras horas. Estos datos sirven para eso. Las reseñas las escriben personas que tienen el juego, y cada una dice cuántos minutos llevaba jugados su autor al escribirla: con eso se sabe si cae dentro o fuera de la ventana de reembolso de dos horas. Por su lado, `appdetails` da lo que cualquiera ve en la tienda antes de pagar, como el precio, el descuento y la nota de la crítica, y de ahí salen las variables del modelo, todas conocidas antes de la compra.

También tienen límites, que la sección 15 desarrolla: las reseñas son en inglés y las escriben compradores de todo el mundo, mientras que los precios son los de la tienda de México y quien usa NexPlay compra desde México.

6. Definición del objetivo

El modelo aprende a ordenar juegos según una señal que sale de sus reseñas: un voto negativo escrito antes de las dos horas de juego. Esa señal no es el arrepentimiento mismo, y su corte de 120 minutos viene de una regla de Steam. Además es rara, y eso obliga a elegir bien la métrica.

6.1. La fórmula

Para cada reseña i ,

$$Y_i = \begin{cases} 1 & \text{si } \text{playtime_at_review}_i < 120 \text{ y } \text{voted_up}_i = 0, \\ 0 & \text{en otro caso,} \end{cases}$$

donde `playtime_at_review` son los minutos que llevaba jugados quien escribió la reseña cuando la publicó y `voted_up` es el pulgar: 1 arriba, 0 abajo⁸. En `data-v1` hay 2,714 reseñas con $Y = 1$ de 123,972, una prevalencia de 2.19 % por reseña⁹. A esa etiqueta se le llama *señal de arrepentimiento temprano*.

6.2. Por qué es una señal proxy

Steam no registra arrepentimiento. Registra que alguien votó en contra y cuánto había jugado al escribir, así que la señal es una proxy¹⁰. Los 120 minutos vienen de la política de reembolsos de la tienda: se puede pedir el reembolso dentro de las dos semanas posteriores a la compra y con menos de dos horas de juego (Valve Corporation, 2024). Una reseña negativa escrita dentro de esa ventana es lo más cercano que dejan los datos a «esto no era para mí, y me di cuenta pronto».

⁸ Al reseñar, Steam pide elegir entre «Recomendado» (pulgar arriba) y «No recomendado» (pulgar abajo).

⁹ La prevalencia es la proporción de casos positivos; aquí, de reseñas con $Y = 1$.

¹⁰ Una variable proxy es una medida que sí se puede observar y que se usa en lugar de otra que no se puede observar directamente.

Por eso el sitio y este documento la llaman señal de arrepentimiento temprano y nunca dicen que alguien se arrepintió. La señal deja fuera a quien se arrepiente y no escribe, a quien pide el reembolso sin reseñar y a quien reseña después de las dos horas. Tampoco mide abandono ni reembolsos: de ninguno de los dos hay dato.

Lo que sí muestran los datos es que las negativas llegan antes. El 15.6 % de las reseñas negativas se escribe antes de los 120 minutos, contra el 2.0 % de las positivas. La señal se queda con ese tramo temprano de las negativas.

6.3. Robustez del umbral de 120 minutos

El umbral no se eligió con los datos: sale de la regla de Steam. Lo que sí se revisó es si moverlo cambia qué juegos tienen más señal. Con 60, 90 o 180 minutos cambia cuántas reseñas cuentan, pero el orden de los juegos casi no se mueve. Su correlación de Spearman con el orden de 120 va de 0.96 a 0.99 (tabla 3)¹¹.

Umbral (min)	Reseñas con señal	Prevalencia por reseña	Spearman con 120 (por juego)
60	1,649	1.33 %	0.964
90	2,250	1.81 %	0.989
120	2,714	2.19 %	1.000
180	3,349	2.70 %	0.993

Tabla 3: La señal con otros umbrales. La prevalencia es por reseña; el Spearman compara las tasas por juego de los 83 juegos contra las de 120 minutos. Fuente: data-v1, con `sensibilidad_umbral` de `backend/analisis/exploracion.py`.

La figura 2 muestra en qué minuto de juego se escriben las reseñas, por separado para las positivas y las negativas. El eje horizontal está en escala logarítmica¹², y cada curva se normaliza sobre su propio grupo para poder compararlas.

En los 120 minutos no hay escalón. Por minuto, las negativas pasan de 11.9 a 12.2 al cruzarlos, y las positivas, de 17.4 a 18.6. El único salto está en los 180 minutos y solo en las positivas, que pasan de 17.6 a 81.7 reseñas por minuto en 81 juegos. Las negativas apenas cambian (de 9.9 a 12.4).

Ese salto no toca la señal, que se define en 120 minutos y con reseñas negativas, y su causa no está en estos datos.

6.4. Por qué PR-AUC

Con una prevalencia de 2.19 %, un modelo que siempre dice «no» acierta el 97.81 % de las veces y no encuentra un solo caso. La exactitud premia justo eso, así que aquí no sirve. El proyecto usa el área bajo la curva de precisión y exhaustividad (PR-AUC, con

¹¹La correlación de Spearman compara dos órdenes, no dos valores: vale 1 cuando los juegos quedan en exactamente el mismo orden con los dos umbrales, aunque sus tasas cambien.

¹²En una escala logarítmica, cada marca del eje multiplica a la anterior por la misma cantidad, por ejemplo por diez. Así caben en una sola gráfica las reseñas escritas a los diez minutos y las escritas después de miles de horas.

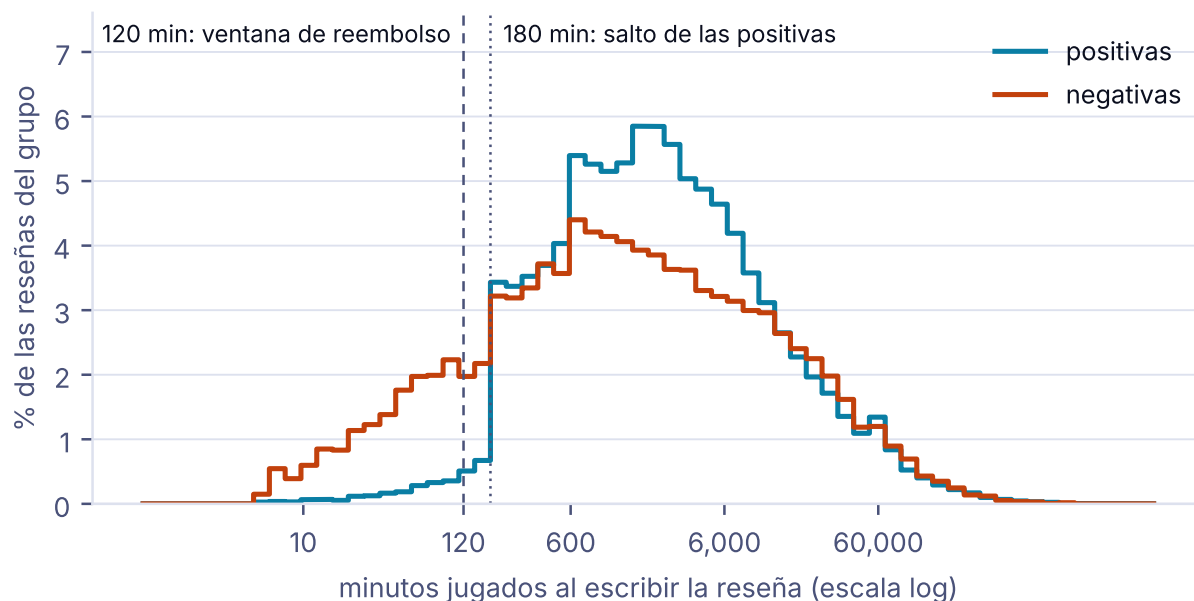


Figura 2: Minutos jugados al escribir la reseña, por pulgar, en escala logarítmica. Cada curva se normaliza sobre su propio grupo. Los intervalos tienen el mismo ancho en escala log, con un borde en 180 y ninguno en 120. Fuente: data-v1.

average_precision_score de scikit-learn)¹³. Mide qué proporción de lo que el modelo pone arriba es de verdad señal, y con clases desbalanceadas informa más que la curva ROC¹⁴ (Davis & Goadrich, 2006; Saito & Rehmsmeier, 2015).

El PR-AUC de un clasificador que no sabe nada es la prevalencia, 0.0219¹⁵. Ese es el piso, y por eso en este documento el PR-AUC del modelo siempre va junto al del trivial. En la validación agrupada por juego¹⁶ el modelo llega a 0.0694, 3.2 veces ese piso (sección 10).

Esos cocientes contra el trivial se redondean con una regla fija. La estimación lleva un decimal, porque el PR-AUC cambia bastante de un fold a otro y una centésima más no diría nada. Los extremos de un intervalo llevan dos, porque ahí lo que importa es qué tan cerca quedan del 1, el valor en el que el modelo no ordena mejor que el trivial.

7. Análisis exploratorio

El notebook 00_exploracion recorre data-v1 con más detalle del que cabe aquí. Esta sección se queda con cuatro hallazgos: los que cambiaron una decisión o la forma de

¹³La precisión es la proporción de los casos que el modelo pone arriba que de verdad tienen la señal; la exhaustividad, la proporción de casos con señal que el modelo encuentra. La curva traza una contra la otra al mover el punto de corte, y el PR-AUC es el área bajo esa curva.

¹⁴La curva ROC compara los aciertos contra las falsas alarmas. Cuando los negativos son muchísimos, las falsas alarmas pesan poco en su cuenta, y la curva puede verse bien aunque el modelo señale muchos casos de más.

¹⁵Ese clasificador trivial ignora las variables y da a todos los casos la misma calificación, así que no ordena nada.

¹⁶La validación agrupada por juego (GroupKFold) reparte juegos completos entre entrenamiento y prueba: ninguna reseña de un juego de prueba se usa para entrenar. Así se mide qué tan bien funciona el modelo con juegos que no vio (sección 9.3).

leer un resultado. Cada uno parte de una pregunta y termina en lo que se hizo con él.

7.1. La tasa varía por juego

¿La señal es parecida en todos los juegos, o depende del juego? La figura 3 ordena los 83 juegos de data-v1 según su tasa de señal, la proporción de sus reseñas con $Y = 1$, y le pone a cada uno su intervalo de Wilson¹⁷. La tasa va de 0.13 % a 27.13 %. Hasta el juego con menos reseñas tiene 682, así que los intervalos son cortos al lado de esa distancia.

La diferencia entre juegos no es ruido: la varianza de la tasa entre juegos es unas 107 veces la esperada si todos tuvieran la misma tasa¹⁸.

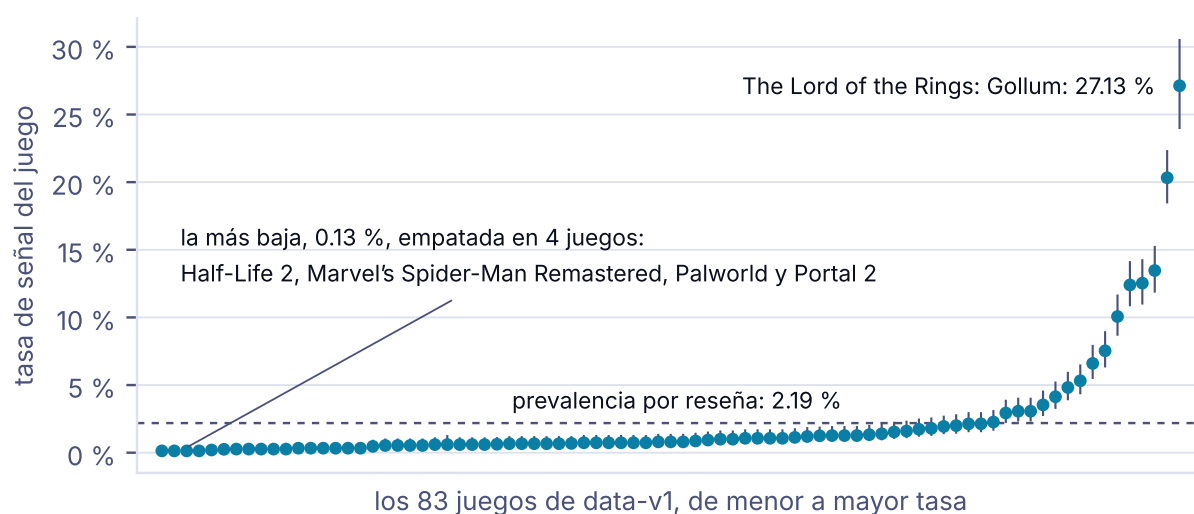


Figura 3: Tasa de señal de cada juego de data-v1, de menor a mayor, con su intervalo de Wilson al 95 %. La línea punteada es la prevalencia por reseña. Fuente: data-v1, con `tasa_por_juego` de `backend/analisis/exploracion.py`.

El otro hallazgo pesa igual. Las cinco variables del modelo (si el juego es gratis, el precio, el descuento, si tiene nota de la crítica y la nota) valen lo mismo en todas las reseñas de un juego, y el notebook lo comprueba juego por juego. El modelo no puede distinguir una reseña de otra del mismo juego: solo puede aprender qué juegos tienen más señal.

De ahí salen dos decisiones. La validación agrupa por juego. Si las reseñas de un juego quedaran a la vez en entrenamiento y en prueba, el modelo se mediría reconociendo juegos que ya vio; la sección 9.3 muestra cuánto infla eso el PR-AUC. Y las variables del juego se analizan con un punto por juego, como en la sección siguiente, porque 123,972 filas que repiten los mismos valores darían una precisión que no existe.

¹⁷Un intervalo de confianza marca entre qué valores es razonable que esté la tasa real, dada la muestra. El de Wilson (1927) se usa con tasas bajas como estas porque, a diferencia del intervalo más común, nunca baja de cero.

¹⁸Si todos los juegos tuvieran la misma tasa, la de cada uno variaría solo por las reseñas que le tocaron. Esa varianza esperada sale de la tasa global y del número de reseñas del juego, como en una binomial. El cociente divide la desviación al cuadrado de cada juego entre su varianza esperada y promedia sobre los juegos: es la ji cuadrada de Pearson entre sus grados de libertad, y vale cerca de 1 cuando solo el muestreo separa a los juegos. Está en la función `sobredispersión` de `backend/analisis/exploracion.py`.

7.2. La crítica acompaña a la señal

¿Qué tienen los juegos con más señal? La tabla 4 compara la tasa de cada juego con cada variable del modelo mediante la correlación de Spearman. Solo la crítica muestra una relación que difícilmente sale del azar¹⁹. Los juegos que tienen nota de Metacritic tienen menos señal (-0.40), y entre ellos, a mejor nota, menos señal (-0.42).

Variable del juego	Juegos	Spearman	p
Precio (juegos de pago con precio)	79	0.18	0.118
Es gratis	83	0.18	0.109
Descuento	83	-0.04	0.731
Tiene nota de la crítica	83	-0.40	< 0.001
Nota de la crítica (solo los que la tienen)	60	-0.42	< 0.001

Tabla 4: Correlación de Spearman entre la tasa de señal de cada juego y cada variable del juego, con su valor p. Fuente: data-v1, con correlaciones_del_juego de backend/analisis/exploracion.py.

La figura 4 lo muestra. A la izquierda están los 23 juegos sin nota, con una tasa mediana de 2.13 %; la de los que sí tienen nota es de 0.73 %. A la derecha, la nube baja conforme sube la nota, aunque con mucha dispersión.

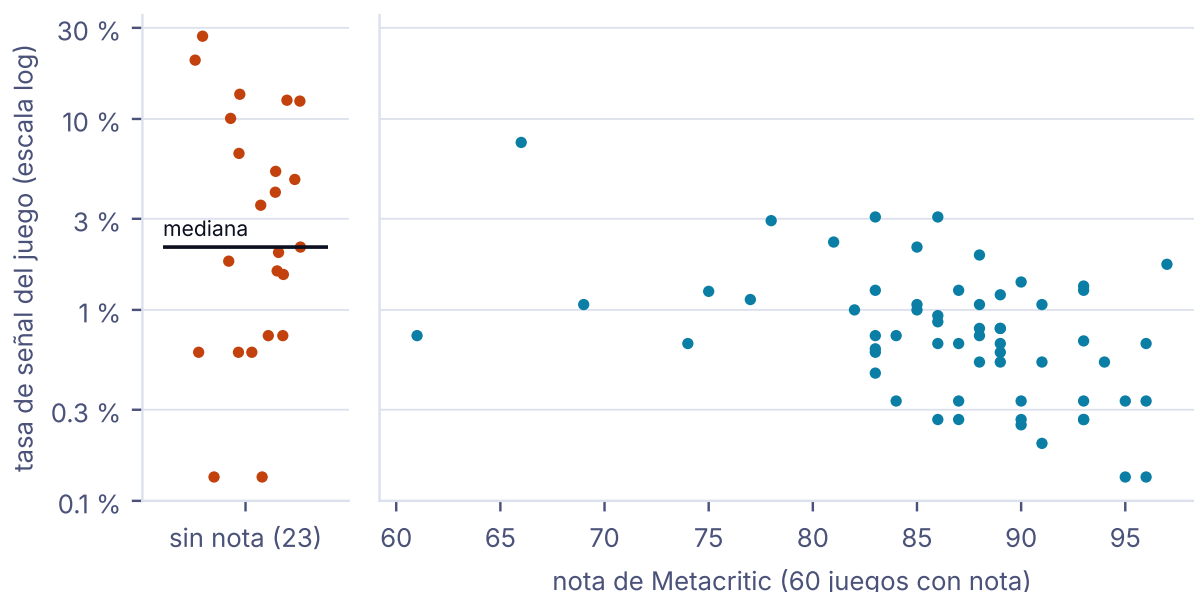


Figura 4: Tasa de señal de cada juego de data-v1 contra su nota de Metacritic, en escala logarítmica. Los juegos sin nota van en un panel aparte, desplazados al azar a lo ancho (con una semilla fija) para que no se encimen; la línea negra es su mediana. Fuente: data-v1.

Con el precio, la relación es débil (0.18, con un valor p de 0.12) y con tan pocos juegos no se distingue del azar. Con el descuento, tampoco. La gratuidad casi no se puede medir, porque en data-v1 solo hay 2 juegos gratis.

¹⁹El valor p es la probabilidad de ver una correlación al menos así de fuerte si en realidad no hubiera ninguna. Con pocos juegos, una correlación débil puede salir por azar, y el valor p lo refleja.

Mirar una variable a la vez puede engañar: si los juegos caros fueran también los que no tienen nota, el precio parecería importar por culpa de la crítica. Por eso también se revisaron las cinco juntas, con los coeficientes del modelo y un intervalo por bootstrap sobre juegos²⁰. Solo los intervalos de la crítica no cruzan el cero, y los dos coeficientes son negativos: tener nota y tener una nota más alta bajan el riesgo estimado del título. El intervalo del coeficiente de tener nota va de -1.04 a -0.55 , y el de la nota, de -0.56 a -0.07 . Los del precio, el descuento y la gratuidad cruzan el cero (sección 9.4).

El modelo se queda con las cinco variables. Todas se conocen antes de comprar y valen igual para cualquier persona. La falta de nota entra como una variable propia, así que el hueco no se rellena en silencio (sección 8). Este hallazgo cambió la explicación: cada factor que muestra el sitio lleva su nivel de evidencia, sólida para la crítica y débil para el precio, el descuento y la gratuidad (sección 11.1).

7.3. El texto de las negativas tempranas

¿Qué escribe quien deja una reseña negativa antes de las dos horas? Esta parte usa el conjunto limpio, sin duplicados exactos ni reseñas vacías (sección 8.2). Ahí hay 105,885 positivas, 14,353 negativas tardías y 2,706 negativas tempranas, que son las que tienen $Y = 1$. Las tempranas son más cortas que las tardías: su mediana es de 22 palabras, contra 27. Las positivas son las más cortas, con 8.

Para encontrar las palabras que separan a dos grupos se usó el log-odds con prior informativo (Monroe et al., 2008)²¹. Frente a las positivas, la palabra que más distingue a las negativas tempranas es «not», y por eso la normalización del texto conserva las negaciones (sección 8.2). Frente a las tardías aparece lo que pasa al empezar a jugar: «tutorial», «account», «login», «settings» y «minutes». Las tardías, en cambio, hablan de «hours», «boss», «mods» y «dlc».

La diferencia más clara es el reembolso. El 9.5 % de las negativas tempranas menciona «refund» o una de sus formas, contra el 2.9 % de las tardías y el 0.2 % de las positivas. Esto encaja con la lectura de la señal como proxy (sección 6.2): quien escribe antes de las dos horas de juego habla de reembolsos mucho más que el resto. No prueba que esas personas lo hayan pedido, y la mayoría de las negativas tempranas no lo menciona.

Este análisis del texto es descriptivo. El texto no entra al modelo de riesgo, que solo usa variables del juego. Quien sí lo lee es la explicación de motivos del sitio, que es una heurística de palabras clave y no un modelo (sección 11.5).

²⁰El bootstrap repite el ajuste muchas veces, cada una con una muestra de juegos sacada al azar y con reemplazo de los 83. El intervalo reúne el 95 % central de los coeficientes que salen. Se remuestran juegos y no reseñas porque lo que varía es el juego. Las variables están estandarizadas, así que el tamaño de los coeficientes se puede comparar entre ellas.

²¹Para cada palabra se compara qué tan seguido aparece en un grupo y en el otro. El prior sale de los dos grupos juntos y evita que una palabra rara parezca muy distintiva solo por aparecer un par de veces. El cálculo está en `log_odds_informativo`, en `backend/analisis/exploracion.py`.

7.4. Los 83 contra los 40

¿Los 40 juegos de la prueba externa se parecen a los 83 del entrenamiento? Pasan las mismas validaciones de llaves y rangos sin un solo caso, pero la mezcla es otra (tabla 5). Hay más juegos gratis (5 contra 2), los de pago son más baratos, con un precio mediano de 269.99 pesos contra 400.00, y la tasa mediana por juego es más alta (1.40 % contra 0.80 %). El promedio por juego, en cambio, queda igual: 2.33 % en los de entrenamiento y 2.33 % en los externos.

	Entrenamiento (data-v1)	Externos
Juegos	83	40
Reseñas	123,972	60,395
Juegos con el tope de 1,500 reseñas	81	40
Juegos gratis	2	5
Juegos sin nota de la crítica	23	10
Nota mediana (los que la tienen)	88	84
Precio mediano de los de pago (pesos)	400.00	269.99
Tasa de señal por reseña	2.19 %	2.34 %
Tasa de señal por juego: mediana	0.80 %	1.40 %
Tasa de señal por juego: promedio	2.33 %	2.33 %

Tabla 5: Los juegos de entrenamiento (data-v1) contra los externos, que son los de data-v2 que no están en data-v1. La tasa por reseña junta todas las reseñas de cada grupo; la mediana y el promedio por juego se calculan sobre la tasa de cada juego. Fuente: releases data-v1 y data-v2.

Con esto no se decidió nada. Los 40 son la prueba externa y nunca entran a elegir variables, a entrenar ni a fijar umbrales. Sí cambia cómo se lee esa prueba: mide al modelo con una mezcla de juegos distinta de la que vio, con más gratis y precios más bajos, así que no repite la validación cruzada. La sección 10.2 da el resultado y la 10.3 explica por qué baja.

8. Procesamiento de datos

El procesamiento tiene dos destinos. El modelo de riesgo se entrena con data-v1 tal como se publicó, porque solo usa variables del juego y no lee texto. La limpieza produce otro conjunto, el limpio, que usan el análisis de texto (sección 7.3) y la medición de los motivos (sección 11.5).

8.1. Validación

Antes de todo, el notebook 00 corre los chequeos de la tabla 2 (sección 5.4). Los de rango que nunca deben fallar, como un minuto negativo o un voto distinto de 0 y 1, son

aserciones²², así que el notebook no sigue si uno falla. También se revisan las copias que cruzan la partición: 57 textos idénticos, de 8 palabras o más, aparecen en más de un juego, y 52 de ellos quedan en folds distintos. Un modelo de texto podría verlos en entrenamiento y en prueba, y por eso la limpieza quita los duplicados. La partición misma está congelada en un archivo (sección 9.3).

8.2. Limpieza: marcar, no borrar

Cada regla es una función de `backend/analisis/limpieza.py`, y se aplican en el orden de la tabla 6. Solo dos quitan filas: los textos idénticos, de los que se queda la reseña más antigua, y las reseñas vacías. Los duplicados quitan 483 filas y las vacías 545, y quedan 122,944 reseñas. Las demás reglas agregan una columna que marca la reseña y la dejan en su lugar. Las cortas, por ejemplo, son el 22.7 % de las reseñas, y quitarlas cambiaría qué reseñas se estudian, no solo cuánto ruido hay. La normalización pasa el texto a minúsculas, quita el BBCode²³ y las direcciones web, y abre las contracciones: «don't» queda como «do not», y la negación, como palabra suelta.

Regla, en orden	Qué se hace	Afectadas	Quedan
Texto idéntico de 8 palabras o más	se quitan; queda la más antigua	483 (0.4 %)	123,489
Reseña vacía	se quitan	545 (0.4 %)	122,944
Menos de 3 palabras	se marcan	27,944 (22.7 %)	122,944
Otro idioma, según lengua	se marcan	1,637 (1.3 %)	122,944
Plantilla de casillas	se marcan	191 (0.2 %)	122,944
Normalización del texto	se agrega <code>texto_norm</code>	94,303 (76.7 %)	122,944

Tabla 6: Bitácora de la limpieza sobre `data-v1`. El porcentaje es sobre las reseñas que llegan a cada regla; en la normalización, las afectadas son las que cambian de texto. Fuente: `limpiar de backend/analisis/limpieza.py`.

La limpieza no se aplica al modelo de riesgo. Se midió qué le pasaría con la misma partición congelada, para que la diferencia saliera de las filas y no del reparto de los juegos. Sin los duplicados, su PR-AUC pasa de 0.0694 a 0.0695, y sin las vacías, a 0.0696. Sin las cortas subiría a 0.0800, pero sobre otra población y con más prevalencia, así que los dos números no se comparan. El modelo se queda con `data-v1` tal cual. El conjunto limpio se guarda con un sha256 de su contenido, y quien lo rehaga puede comprobar que obtuvo el mismo.

²²Una aserción es una comprobación dentro del código: si no se cumple, la ejecución se detiene con un error.

²³El formato que Steam permite en las reseñas, con etiquetas como `[h1]` o `[spoiler]`.

8.3. Idioma

La ingesta pidió reseñas en inglés (`language=english`), y Steam entrega las que llevan esa etiqueta. Para revisarla se usó `lingua`²⁴: el 98.7 % de las reseñas limpias está en inglés o no se pudo determinar, y 1,637 están escritas en otro idioma, sobre todo ruso, español y portugués. Como el filtro lo aplicó Steam, esa cifra mide qué tan bien etiquetaron sus reseñas los autores, no qué idiomas se usan en la tienda. Las reseñas en otro idioma se marcan y se quedan. Al modelo de riesgo no le afectan, porque no lee texto, pero a los motivos sí, porque sus palabras clave están en inglés (sección 11.5).

8.4. Precio imputado

Los 2 juegos de pago que llegaron sin precio (tabla 2) entran al modelo con precio 0: la variable es el logaritmo de uno más el precio, y los faltantes se llenan con 0. El modelo los lee como si costaran 0, aunque no los marca como gratis, y como el coeficiente del precio es positivo, eso tiende a bajar su riesgo estimado. La ficha de esos juegos lo avisa (sección 11.4).

8.5. Variables que no se conocen al comprar

Una variable solo puede entrar al modelo si se conoce antes de comprar. La tabla 7 repasa las candidatas. Las del juego vienen de la tienda y entran. Las del autor de la reseña, `no.` `num_reviews` cambia con el tiempo y se lee al descargar, y `steam_purchase`, `received_for_free` y `written_during_early_access` solo existen porque la reseña ya se escribió: ninguna está disponible al momento de la compra. La biblioteca del autor sí podría declararse antes; se probó, y su aporte cupo en el ruido entre folds (sección 9.2). En el sitio, el perfil se declara en un formulario y sirve para la afinidad: no mueve el riesgo.

El texto de la reseña es otro caso. Tampoco existe al comprar, pero además sería una fuga²⁵: la señal Y sale de esas mismas reseñas, de su voto y de sus minutos, y el texto acompaña a ese voto. Por eso solo lo leen la exploración (sección 7.3) y los motivos (sección 11.5).

²⁴Una biblioteca de detección de idioma. Solo decide cuando tiene confianza; los textos muy cortos o sin sentido quedan como indeterminados.

²⁵Hay fuga cuando las variables de un modelo ya contienen la respuesta que debe predecir. Sus métricas salen mejores de lo que serán en uso.

Variable	Cuándo se conoce	¿Entra al modelo de riesgo?
es_gratis, precio (en logaritmo) y descuento	en la tienda, antes de comprar	sí
Si tiene nota de Metacritic, y la nota	en la tienda, antes de comprar	sí; la nota que falta se rellena con la mediana, junto a la bandera
num_games_owned: juegos del autor	en la reseña; en el sitio se declara «compras al año»	no: su aporte cabe en el ruido entre folds, y 0 es un perfil privado
num_reviews: reseñas del autor	al descargar; cambia con el tiempo	no: no está disponible al momento de la compra
steam_purchase, received_for_free y written_during_early_access	en la reseña	no: no están disponibles al momento de la compra
playtime_at_review y voted_up	en la reseña	no: definen Y
Texto de la reseña	en la reseña	no: sería una fuga, porque Y sale de esas mismas reseñas

Tabla 7: Variables candidatas: cuándo se conocen y si entran al modelo de riesgo. Fuente: `construir_features` de `backend/modelado/entrenar_baseline.py` y la sección «Variables del jugador» del notebook 00.

9. Modelación

9.1. El modelo y sus conjuntos de variables

El modelo es una regresión logística²⁶ con las variables estandarizadas y pesos de clase balanceados²⁷, sin ajuste de hiperparámetros. La misma función, `construir_pipeline`, arma el modelo que sirve la API y el de los notebooks. Se probó con tres conjuntos de variables, siempre contra el clasificador trivial de la sección 6.4 (tabla 8).

El conjunto completo da el PR-AUC más alto, 0.0849, pero no se puede usar: sus variables posteriores a la reseña no están disponibles al momento de la compra (sección 8.5). La comparación que importa es entre juego y compra.

9.2. La decisión B+: no forzar una personalización que los datos no justifican

El proyecto se quedó con el conjunto juego, la variante que llamó B+, por dos razones.

La primera es estadística. Compra le agrega al conjunto juego la biblioteca del autor, lo único del jugador que el sitio podría pedir en un formulario. Con eso, el PR-AUC sube 0.0015, muy por debajo de la desviación entre folds del modelo (0.0415), y compra le

²⁶Un modelo lineal de clasificación: le da un peso a cada variable, las suma y convierte el resultado en un score entre 0 y 1.

²⁷Con `class_weight="balanced"`, las pocas reseñas con $Y = 1$ pesan en el ajuste tanto como todas las demás juntas, para que la clase rara no se pierda.

Conjunto	Variables	Cuántas	PR-AUC	Desv. entre folds
Trivial	ninguna	0	0.0219	0.0037
Juego (el del sitio)	gratuidad, precio, descuento y crítica	5	0.0694	0.0415
Compra	las de juego y los juegos del autor	6	0.0709	0.0270
Completo	las de compra, la bandera de perfil privado y las posteriores a la reseña	11	0.0849	0.0207

Tabla 8: PR-AUC del mismo modelo con cada conjunto de variables: media y desviación estándar de los 5 folds de la partición congelada. Fuente: `evaluar_gkf` de `backend/modelado/entrenar_baseline.py` sobre `data-v1`.

gana a juego en 3 de los 5 folds. Con 30 particiones distintas de los juegos, compra gana en 16: más o menos la mitad de las veces, como al azar.

La segunda es de diseño. Cuando el sitio servía el conjunto compra, la variable `log_num_games_owned` se llenaba con las compras al año que la persona declara en el formulario, pero el modelo la había aprendido del tamaño de la biblioteca de quien escribió la reseña. Las compras declaradas por año eran un sustituto de ese dato que nunca se validó.

Con el conjunto juego, el riesgo es del título y vale igual para cualquier persona: no depende de un dato que nadie verifica y se explica solo con propiedades del juego. El perfil que se declara en el sitio sirve para la afinidad, que dice qué tanto encaja un juego con quien lo declaró, y no mueve el riesgo.

9.3. Validación agrupada por juego

La sección 7.1 explica por qué se agrupa por juego. Si las reseñas se repartieran al azar entre folds, el mismo modelo daría 0.0799, con una desviación de apenas 0.0032, porque cada fold de prueba tendría juegos que el modelo ya vio. Agrupando por `appid` da 0.0694 ± 0.0415 , y esa es la cifra que vale para un juego nuevo.

El reparto de los juegos también pesa: con 20 particiones por juego hechas al azar, el PR-AUC medio va de 0.0477 a 0.0873. Una diferencia menor que ese rango no dice nada de los datos, y por eso la partición está congelada en un archivo de `backend/referencias/`: toda evaluación y los umbrales la leen de ahí. Hay otra razón. `GroupKFold` reparte los juegos según su número de reseñas, y 73 de los 83 tienen exactamente el tope de 1,500. Cada versión de `scikit-learn` desempata distinto: con la 1.6.1, la que trae Colab, solo 12 juegos quedan en el mismo fold. Aun con esa otra partición, el PR-AUC es 0.0692 ± 0.0412 y el modelo le gana al trivial en 5 de 5 folds, igual que con la congelada.

9.4. Coeficientes

Los coeficientes con su intervalo por bootstrap sobre juegos están en la sección 7.2: solo los dos de la crítica no cruzan el cero. Los demás sí. El del precio va de -0.01 a 0.94 , el del descuento, de -0.31 a 0.07 , y el de la gratuidad, de -0.12 a 0.79 . El del precio es positivo en el 96.6 % de las réplicas, pero con 83 juegos su intervalo no descarta el cero. Por eso su evidencia se marca como débil (sección 11.1).

9.5. Umbrales de riesgo

El sitio no muestra el score: muestra una de tres bandas, bajo, medio o alto. Los cortes dividen en tercios los scores fuera de pliegue²⁸ de las reseñas de data-v1: un juego pasa a medio desde 0.2858 y a alto desde 0.3916 . Los tercios son de reseñas, y cada juego se califica con el modelo entrenado con todo data-v1, así que los juegos no se reparten en partes iguales: los 83 quedan en 30, 23 y 30, y los 123 del catálogo, en 43, 37 y 43.

Por los pesos balanceados, el score no es una probabilidad calibrada: los cortes quedan muy por encima de la prevalencia, que es de 2.19 %. Sirve para ordenar títulos, no como la probabilidad de que una compra termine en la señal de arrepentimiento temprano. Los umbrales salen solo de data-v1, y los 40 externos se clasifican con ellos sin ajustarlos.

9.6. Modelos de texto (Parte A)

Los modelos que leen el texto de las reseñas se están haciendo en la rama `modelos-texto`, con prerregistro²⁹. Su resultado entrará a este documento cuando esa rama llegue a master. **[PENDIENTE DE EVIDENCIA]**

10. Evaluación y generalización

El modelo se evalúa dos veces: con la validación agrupada sobre los 83 juegos de entrenamiento y con los 40 que nunca vio. En las dos, su PR-AUC se lee junto al del trivial (sección 6.4).

10.1. Resultados por fold

La figura 5 muestra el PR-AUC de cada fold de la partición congelada. El modelo le gana al trivial en 5 de 5 folds, pero por márgenes muy distintos: de 1.8 a 5.8 veces. En promedio llega a 0.0694 contra 0.0219 , 3.2 veces el trivial, con una desviación entre folds de 0.0415 . Es una desviación grande al lado de la media: el resultado depende de qué juegos le tocan a cada fold, como ya se vio en la sección 9.3.

²⁸El score fuera de pliegue de una reseña lo da un modelo entrenado sin el juego de esa reseña, con la misma partición congelada.

²⁹Un prerregistro fija, antes de correr el análisis, qué se va a medir y qué resultado contará como éxito, para no acomodar la pregunta a los resultados.

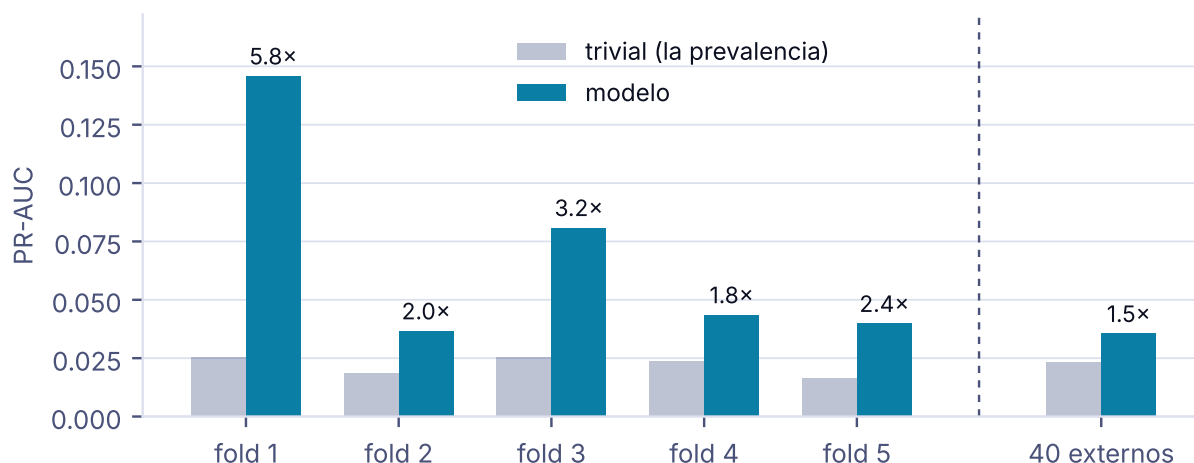


Figura 5: PR-AUC del modelo y del trivial en cada fold de la partición congelada (data-v1) y en la prueba externa. Sobre cada barra, cuántas veces supera el modelo al trivial. Fuente: evaluar_gkf sobre data-v1 y docs/evidencia/prueba-externa.json.

10.2. Prueba externa en 40 títulos

Los 40 títulos que llegaron con data-v2 se califican con el modelo entrenado en data-v1, con sus umbrales y su mediana de Metacritic, igual que lo hace la API. Su PR-AUC es 0.0356 contra 0.0234 del trivial: 1.5 veces, la última pareja de la figura. Para saber qué tan firme es esa ventaja, los títulos se remuestrearon 2,000 veces con reemplazo. El intervalo al 95 % del cociente va de 1.03 a 2.33, y solo 27 réplicas quedan en 1 o menos. Hay señal fuera de muestra, pero es modesta: el extremo inferior queda cerca de 1.

Entrenar con los 123 juegos juntos bajaría el ruido entre folds, pero se perdería la única evaluación con juegos que el modelo no vio. Por eso el modelo se queda entrenado con data-v1.

10.3. Por qué cae fuera de muestra

Una parte de la caída se ve en la variable que más pesa en el modelo, la crítica (sección 7.2): de los 43 juegos de riesgo alto del catálogo, 33 no tienen nota. La tabla 9 da la tasa real por banda y cobertura de crítica en los dos cortes. En los de entrenamiento es solo descriptiva, porque el modelo ya los vio; los externos son los que evalúan algo.

Por reseña, en los de entrenamiento la banda alta sin nota tiene 5.37 % contra 0.66 % de la baja. En los externos la distancia se acorta: 3.66 % contra 1.86 %. La falta de nota sigue señalando más riesgo, pero separa menos en los títulos nuevos, y la baja y la media se invierten (1.86 % contra 1.63 %). Con el promedio por juego la lectura es la misma: 3.64 % contra 1.86 % en los externos. La fila de alto con nota junta solo 10 juegos entre los dos cortes, así que es una pista y no una conclusión. Además, los externos traen otra mezcla de juegos (sección 7.4).

Banda	Crítica	Juegos	Tasa por reseña	Promedio por juego
<i>Los 83 de entrenamiento (descriptivo)</i>				
bajo	con nota	30	0.66 %	0.66 %
medio	con nota	23	1.11 %	1.11 %
alto	con nota	7	2.18 %	2.19 %
alto	sin nota	23	5.37 %	5.78 %
<i>Los 40 externos</i>				
bajo	con nota	13	1.86 %	1.86 %
medio	con nota	14	1.63 %	1.61 %
alto	con nota	3	3.31 %	3.31 %
alto	sin nota	10	3.66 %	3.64 %

Tabla 9: Tasa de señal por banda de riesgo y cobertura de crítica. Fuente: releases data-v1 y data-v2 con las bandas de backend/referencias/bandas_referencia.json; replica docs/evidencia/metacritic-por-banda.md.

10.4. Casos al filo del corte

Las bandas cortan un score continuo, y un juego cerca de un corte puede cambiar de banda con un cambio mínimo en los datos o en el reentrenamiento. Los dos más cercanos son Hollow Knight, con 0.2857 contra el corte de 0.2858, que queda en la banda baja, y Warframe, con 0.3918 contra 0.3916, que queda en la alta. Por eso no se usan como casos de demostración, y la ficha acompaña la banda con los factores que la explican (sección 11.1).

11. Interpretabilidad

El sitio no se queda en la banda. Dice qué la mueve y de qué se quejan las reseñas con la señal. Esta sección explica cómo se arma cada pieza y hasta dónde llega.

11.1. Factores por aporte y su evidencia

Para cada juego, la API calcula el aporte de cada variable al score: su coeficiente por su valor estandarizado, que es lo que suma la regresión logística³⁰. La ficha muestra los tres factores de mayor aporte, en ese orden, aunque el primero tenga evidencia débil. Cada uno lleva su nivel de evidencia (tabla 10): sólida para la cobertura y la nota de la crítica, y débil para la gratuidad, el precio y el descuento, según sus intervalos por bootstrap (sección 9.4). La ficha lo dice con palabras: «evidencia débil: con 83 juegos no se distingue de cero».

³⁰El aporte está en log-odds, la escala en la que la regresión suma: positivo sube el score y negativo lo baja.

Factor en la ficha	Evidencia	Contra qué se lee
cobertura de crítica especializada	sólida	si Steam muestra una nota
nota de Metacritic	sólida	el promedio del catálogo, 85.5
gratuidad del juego	débil	si es gratis; en los gratis suma el aporte del precio
precio del juego	débil	la mediana de los de pago, 349.88 pesos
descuento actual del juego	débil	si hay descuento

Tabla 10: Los factores de la ficha, su evidencia y la referencia contra la que se lee cada uno. Fuente: backend/api/scoring.py y los juegos de data-v3.

11.2. Lectura contra el catálogo

Junto a cada factor va la cifra del juego y la referencia contra la que se lee. La nota se compara con el promedio de los juegos del catálogo que la tienen, 85.5, y el precio con la mediana de los de pago, 349.88 pesos. Para el precio se usa la mediana porque el modelo trabaja con su logaritmo, y en esa escala la referencia se parece más a la mediana que al promedio, que los juegos caros inflan.

11.3. Lo típico del catálogo: una decisión de producto

Un factor cuyo aporte es menor que UMBRAL_TIPICO, 0.10, se muestra sin flecha y con la leyenda «cerca de lo típico del catálogo; casi no mueve la estimación». Ese valor es una decisión de producto, no una propiedad estadística: no sale de un intervalo ni de una prueba. Se eligió para que ninguna flecha contradiga la cifra que tiene al lado. Hace falta porque el modelo compara cada nota contra su media de entrenamiento, 86.97, mientras que la ficha la compara contra el promedio del catálogo, 85.5. Una nota entre las dos aparece «por encima del promedio» y, aun así, recibe un aporte pequeño que sube el score. Con 0.10, ningún factor con flecha cae en ese caso en los 123 juegos del catálogo; el generador de este documento lo comprueba.

11.4. Avisos

Dos casos llevan aviso. En un juego gratis, la gratuidad y el precio de 0 dicen lo mismo y tiran en sentidos opuestos, así que se muestran como un solo factor, con los dos aportes sumados. El aviso agrega que en el entrenamiento había solo 2 juegos gratis y que, para ellos, el modelo extrapola. En los 2 juegos de pago sin precio (sección 8.4), el aviso dice que la estimación es menos confiable, porque el modelo tomó el precio como 0 y eso tiende a bajar su riesgo estimado.

11.5. Los motivos: una heurística de palabras clave

/explicacion no usa un modelo. Busca palabras clave en inglés de 6 categorías (rendimiento, bugs, dificultad, controles, contenido y precio) en las reseñas con la señal de cada juego, y una reseña puede caer en varias o en ninguna. Si el juego tiene menos de 5 reseñas con la señal, no muestra motivos. Cada porcentaje se calcula sobre las reseñas que sí cayeron en alguna categoría, y la ficha dice cuántas son.

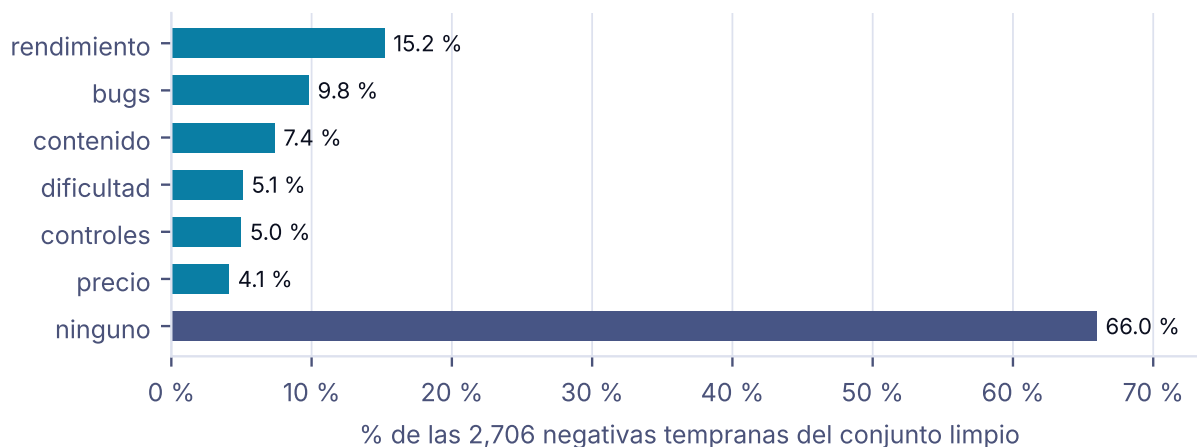


Figura 6: Negativas tempranas del conjunto limpio de data-v1 que mencionan cada motivo de la lista de palabras clave, como porcentaje de las 2,706. Una reseña puede mencionar varios. Fuente: `backend/analisis/motivos.py`, como en la sección «Motivos por palabras clave» del notebook 00.

El límite de esta heurística es su cobertura. De las 2,706 negativas tempranas del conjunto limpio, el 34.0 % menciona al menos un motivo de la lista; el 66.0 % restante no menciona ninguno (figura 6). Rendimiento y bugs son los más frecuentes. Por eso los motivos se leen como las quejas de las reseñas que usan esas palabras, no como la causa de la señal en ese juego.

12. Arquitectura y despliegue

12.1. De los datos a la interfaz

[PENDIENTE: redacción]

12.2. Verificaciones en el build

[PENDIENTE: redacción]

12.3. Endpoints

[PENDIENTE: redacción]

12.4. Costos y límites

[PENDIENTE: redacción]

13. Interfaz de uso

13.1. Las cinco vistas

[PENDIENTE: redacción]

13.2. Paso a paso de una consulta

[PENDIENTE: redacción]

13.3. Nía

[PENDIENTE: redacción]

13.4. Pruebas

[PENDIENTE: redacción]

14. Resolución del problema

[PENDIENTE: redacción]

15. Limitaciones

[PENDIENTE: redacción]

16. Conclusiones

[PENDIENTE: redacción]

17. Siguiendo pasos

[PENDIENTE: redacción]

Referencias

- Davis, J., & Goadrich, M. (2006). The Relationship Between Precision-Recall and ROC Curves. *Proceedings of the 23rd International Conference on Machine Learning*, 233-240. <https://doi.org/10.1145/1143844.1143874>
- Monroe, B. L., Colaresi, M. P., & Quinn, K. M. (2008). Fightin' Words: Lexical Feature Selection and Evaluation for Identifying the Content of Political Conflict. *Political Analysis*, 16(4), 372-403. <https://doi.org/10.1093/pan/mpn018>
- Saito, T., & Rehmsmeier, M. (2015). The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLOS ONE*, 10(3), e0118432. <https://doi.org/10.1371/journal.pone.0118432>
- Valve Corporation. (s.f.-a). *IUserReviewsService*. Consultado el 30 de septiembre de 2026, desde <https://partner.steamgames.com/doc/webapi/IUserReviewsService>
- Valve Corporation. (s.f.-b). *User Reviews – Get List* [Consultada el 30 de septiembre de 2026; la página marca el endpoint como obsoleto y remite a *IUserReviewsService*]. Consultado el 30 de septiembre de 2026, desde <https://partner.steamgames.com/doc/store/getreviews>
- Valve Corporation. (2024, 23 de abril). *Steam Refunds*. Consultado el 30 de septiembre de 2026, desde https://store.steampowered.com/steam_refunds/
- Wilson, E. B. (1927). Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22(158), 209-212. <https://doi.org/10.1080/01621459.1927.10502953>

A. Reproducibilidad

[PENDIENTE: redacción]